

Implementation Plan

Document 06

Status	Draft
Version	1.4 · 2026-10-02 02:38 · change log
Team	1 developer/reviewer. CPU laptop + Kaggle GPU
Related	All previous docs. Each task ID maps to PRD/TRD requirement IDs where applicable

Contents

1	Pilot: one recording before Phase 0 (done)	2
2	Phase 0: Foundations (≈5 days)	3
3	Phase 1: Alignment pipeline (≈7 days)	3
4	Phase 2: Review UI v2 (≈6 days)	4
5	Phase 3: Datasets + zero-shot baselines (≈5 days)	4
6	Phase 4: Fine-tune v1 on Kaggle (≈6 days, GPU-quota bound)	5
7	Phase 5: Transcribe service (≈8 days; can overlap with P4)	5
8	Phase 6: Feedback loop + model v2 (≈10 days)	6
9	Definition of done (every task)	6
10	Experiment log template (docs/experiments/EXX_name.md)	6
11	Risks to the schedule	7
12	Immediate next steps	7
	Change log	8

The phases are ordered so that each one produces something usable and measurable. Estimates are working days for one person. Review hours for labelling are listed separately because they run in parallel with coding.

Task tracking

Changed in v1.3: see the [change log](#).

This document holds the tasks and their acceptance criteria; **who does what and the current status live in [GitHub Issues](#)**, so the document does not need a new version for every finished task. Labels: owner:you (assigned to you), owner:claudes, and phase:pilot, phase:0 ... phase:6.

Plan item	Issue
Pilot on Kaggle (you)	#1
Recording names, speaker groups (you)	#2
Choose test_v1 / dev_v1 (you)	#3

Plan item	Issue
P0.1–P0.9	#4–#12, one per task
Phases 1–6	#13–#18, one tracking issue per phase with a task checklist

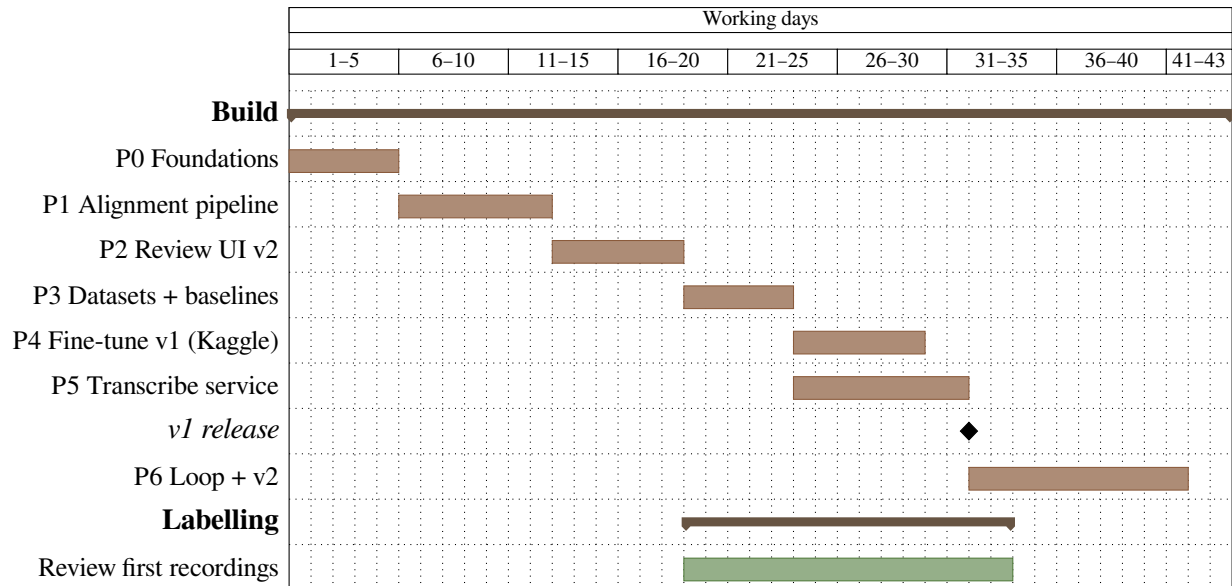


Figure 1: Indicative schedule for one developer (working days).

1 Pilot: one recording before Phase 0 (done)

Changed in v1.2: see the change log.

Before building the full system we measured one real recording, NRCCW_KSM09 (2 h 05 min, 1 237 turns), to learn how well its transcript aligns and how stock Whisper does. The pilot lives in `src/transcribe/pilot/` and becomes the alignment and evaluation core of Phases 1 and 3.

Built	What it does
<code>pilot.run</code>	DOCX + audio → alignment (TRD §4.3), 2–30 s segments with flags, zero-shot Whisper (WER, CER, per-language WER, switch-point errors, words in untranscribed gaps), <code>report.md/json</code> , optional review chunks
<code>pilot.batch</code>	Runs every audio/DOCX pair in a folder and writes <code>summary.md</code>
Kaggle runbook	<code>scripts/kaggle_pilot.sh</code> + <code>notebooks/kaggle_pilot.ipynb</code> ; see 07 Kaggle Runbook
<code>pilot.make_mock</code>	Known-answer smoke test from public FLEURS clips
<code>pilot.review_stats</code>	Review speed from the review app's save timestamps
Tests	24 tests in <code>tests/pilot</code>

Results on NRCCW_KSM09 (alignment): words with timings 11 590 of 12 305; 1.70 h of segments, 0.91 h clean; no word more than 30 s from a transcriber timestamp; held-out median error 1.3 s. The laptop CPU computes emissions at about $0.8\times$ real time and runs Whisper small at about $2\times$ real time, so heavy runs move to Kaggle.

Also done early: P0.3 (Transcribe no longer overwrites reviewer text) and the R: speaker-label fix.

Still open: zero-shot Whisper numbers on correctly aligned segments (the first run used the old alignment), and the same pilot on the other 10 recordings. Both come from one Kaggle run.

2 Phase 0: Foundations (≈5 days)

Goal: a clean base that stops the known bugs from spreading into new work.

ID	Task	Acceptance
P0.1	Make transcribe an installable package: drop the <code>sys.path</code> hack and the <code>src.</code> imports; entry points <code>transcribe-api</code> , <code>transcribe-worker</code> , <code>transcribe-cli</code>	<code>pip install -e .</code> then <code>python -c "import transcribe"</code> works; all scripts run via <code>python -m transcribe...</code>
P0.2	Move <code>models/</code> , <code>training/</code> , <code>data/</code> , <code>script/{train,evaluate,infer}.py</code> into <code>transcribe/legacy/</code> unchanged	Legacy train still runs on a toy dataset
P0.3	Fix: <code>transcribe_current</code> must not overwrite <code>segment["transcript"]</code> ; store the hypothesis only	Test: after Transcribe, the transcript equals the previous value
P0.4	Fix: grouped split by recording in the legacy <code>prepare_dataset</code> ; build the vocabulary from train only	Test: no recording in two splits
P0.5	Add <code>pytest</code> , <code>ruff</code> , and a GitHub Actions workflow (Ubuntu, Py 3.10)	CI green on the branch
P0.6	SQLAlchemy models + Alembic initial migration from 05 Backend Schema	<code>alembic upgrade head</code> creates all tables; invariant tests 1, 2, 8 pass
P0.7	<code>transcribe.db.migrate_json</code> imports the existing <code>data/projects/*</code> manifests	Approved segments are preserved; candidates are dropped with a count printed
P0.8	Extend <code>docs/STYLE_GUIDE.md</code> (v0.1 drafted) (verbatim rules, hybrid words, numbers, markers, language tags) with 20 worked examples from real transcripts	Reviewed by you; referenced from the UI help
P0.9	Split requirements into <code>requirements/*.txt</code> as in the TRD §7	Fresh <code>venv</code> installs <code>core+corpus+ui</code> on Windows and Linux

3 Phase 1: Alignment pipeline (≈7 days)

Changed in v1.2: see the [change log](#).

Goal: DOCX + audio in, pre-filled segments out.

ID	Task	Acceptance
P1.1	<code>corpus/ingest.py</code> : <code>sha256</code> , <code>ffmpeg</code> normalise to 16 kHz mono, store the original	Re-importing the same file → 409
P1.2	Move the parsers into <code>corpus/parsers/</code> ; add <code>text_align</code> + <code>word_map</code> generation (<code>corpus/textnorm.py</code>)	Unit tests: annotations removed, number verbalisation (EN via <code>num2words</code> + <code>numbers_sw.py</code> , candidates scored by the aligner, <code>number_check</code> flag on ties), word map round-trips to the raw text
P1.3	<code>corpus/align.py</code> : an Aligner protocol wrapping the pilot's two-pass aligner (<code>pilot/ctc_align.py</code> : relative confidence, anchors, block fill bounded by transcript timestamps)	Pilot numbers hold on every recording in the batch summary: $\geq 90\%$ of words timed, no marker error over 30 s

ID	Task	Acceptance
P1.4	corpus/segment.py: the TRD §3 policy (≤ 30 s, 1.5 s minimum, gap ≥ 300 ms, turn-aware, padding)	Property tests: no segment > 30 s, no word cut, full coverage of aligned words
P1.5	Auto-flags (low_alignment, unclear_marker, overlap_marker) + priority.py	Unit tests on synthetic inputs
P1.6	jobs worker loop with claim, heartbeat and retry; job kinds parse_transcript, align	Killing the worker mid-job $\rightarrow$ the job is re-queued and completes on restart
P1.7	Calibrate thresholds (<i>done in the pilot for NRCCW_KSM09</i>): check the anchor and low_alignment thresholds against transcriber timestamps on each new recording	Marker check in every report.md; held-out median error ≤ 3 s

4 Phase 2: Review UI v2 (≈ 6 days)

Changed in v1.1: [see the change log](#).

Goal: reviewing becomes verification, driven from the keyboard.

ID	Task	Acceptance
P2.1	FastAPI app + the /recordings, /segments, /review/ queue routes; revision endpoint with 409 on conflict	API tests including the conflict case
P2.2	Web shell (plain HTML/JS served by FastAPI) with 3 sections and theme tokens from 03 UI/UX §2. The Transcribe section exists (transcribe.app)	Loads at 127.0.0.1:8000
P2.3	Import wizard + recording overview with confidence timeline	Import $\rightarrow$ align $\rightarrow$ overview with no CLI
P2.4	Segment reviewer: pre-filled text, flags, boundary nudging with slice re-render, approve & next, history/restore	A reviewer can complete 20 segments using only the keyboard
P2.5	Hotkey JS layer	All keys in the §4.5 table work
P2.6	Language span tagging (P1 priority; may slip to P6)	Spans persist per revision
P2.7	Measure throughput on one real recording: minutes of review per audio minute vs. the old type-from-scratch UI	Recorded in docs/experiments/E2_review_speed.md; target $\geq 3\times$ faster

Speaker grouping (before choosing test/dev): a one-off script, corpus/speaker_groups.py, embeds each recording's speakers (ECAPA-TDNN, CPU) and lists cross-recording pairs with high similarity. You confirm them, combine them with the naming convention, and the groups are stored so the dataset builder never splits a group.

Labelling track (parallel, starts at the end of P2): review the current 11 recordings (8 h 56 min, ≈ 13 –14 h of review at $1.5\times$ real time): **test_v1 = 2 recordings (≈ 1.5 h)** and **dev_v1 = 1 recording (≈ 50 min)** first, then **train = 8 recordings (≈ 6.5 h)**. The long-run targets (test ≈ 3 h / ≥ 15 recordings, train ≥ 20 h) are reached as new audio arrives. Review the test/dev recordings first; they matter most and must be the most carefully checked.

5 Phase 3: Datasets + zero-shot baselines (≈ 5 days)

Goal: know where we stand before training anything. This is the most instructive phase for learning the process.

ID	Task	Acceptance
P3.1	datasets/build.py: filter spec, grouped split, frozen test_v1, sha256, invariants 4–7	Tests for leakage and consent enforcement
P3.2	datasets/export.py: audiofolder export + dataset card	load_dataset("audiofolder", ...) loads it on Kaggle
P3.3	evaluation/normalizer.py + metrics.py (WER, CER, per-language, switch-point, hallucination) with hand-computed tests	100% test coverage on metric functions
P3.4	inference/engine.py (faster-whisper) + register the stock models as zero-shot baselines	whisper-small, whisper-medium and large-v3-turbo are registered
P3.5	Experiment E3 (zero-shot): evaluate each stock model on dev with language sw, en and auto	A table in docs/experiments/E3_zero_shot.md with an error analysis of 30 worst segments (translation? hallucination? spelling of hybrids?)

6 Phase 4: Fine-tune v1 on Kaggle (≈6 days, GPU-quota bound)

ID	Task	Acceptance
P4.1	finetune/whisper_train.py: HF Seq2SeqTrainer, config YAML, fixed < sw > token, SpecAugment, non-speech samples, public-data mixture with weights, resume support, run_manifest.json	Dry run on 50 samples on CPU completes 5 steps
P4.2	Kaggle notebook notebooks/kaggle/finetune_whisper.ipynb (thin wrapper) + a private dataset upload of the source package	Runs end to end on a T4
P4.3	Experiment E4: whisper-small full fine-tune, in-domain only vs. in-domain + public mixture	Dev WER for both; the winner is kept
P4.4	Experiment E5: language token sw vs en vs none (on the E4 winner, 1.5k steps each)	Decision recorded
P4.5	Register + CT2 int8 convert + evaluate on test_v1	whisper-small-kct-v1 has a test report; gate: clean test WER ≤ 20% or a documented plan for what to change
P4.6	(Learning track, optional) finetune/w2vbert_ctc_train.py + a KenLM 4-gram trained on approved train text	Comparison row in the evaluation table

7 Phase 5: Transcribe service (≈8 days; can overlap with P4)

Goal: the v1 product surface.

ID	Task	Acceptance
P5.1	inference/vad.py (Silero), longform.py (windowing ≤ 30 s), guards.py (no-speech, compression ratio, n-gram loop)	A 10 min music-only file produces ≤ 10 words
P5.2	transcribe job with incremental segment writes and progress/ETA	A 1 h file on a 4-core CPU finishes with RTF ≤ 0.5
P5.3	Transcribe tab: upload, jobs list, editor with autosave, low-confidence highlighting	Manual walkthrough of a 20 min file
P5.4	Exports TXT/SRT/VTB/JSON (+ DOCX P1); edited text wins over raw output	Golden-file tests for each format

ID	Task	Acceptance
P5.5	Model registry screen, promotion gate, hot reload	Promoting v1 switches Transcribe without a restart
P5.6	Packaging: Dockerfile (CPU), make run, a first-run setup checklist (ffmpeg, model weights)	Fresh machine to a working app in < 15 min following the README

v1 release = end of P5. A working prototype: upload, transcribe, edit and export, with a fine-tuned model and a test report.

8 Phase 6: Feedback loop + model v2 (~10 days)

ID	Task	Acceptance
P6.1	“Promote to corpus” from Transcribe (consent-checked, edited segments only)	Invariant tests pass
P6.2	VAD-only import for untranscribed recordings + model pre-fill	Flow B works end to end
P6.3	Active-learning priority from model uncertainty; weekly batch job	The queue orders by uncertainty
P6.4	Domain eval sets: ≥ 1 h each of TV and church audio, reviewed, stored as external test sets	Per-domain WER in reports
P6.5	Experiment E6: whisper-large-v3-turbo with LoRA (r=32, attention + FFN) vs. v1	Promoted only if it passes the gate; RTF is checked on CPU
P6.6	Learning curve plot: test WER vs. training hours across snapshots	docs/experiments/E6_learning_curve.md
P6.7	Optional: pyannote diarization for Speaker 1/2 labels in the Transcribe output	Behind a feature flag

9 Definition of done (every task)

- Code under src/transcribe/... with type hints. Unit tests are added, and CI is green.
- No change to reviewer text by any automated path (invariant 3 test stays green).
- Any new metric or experiment result is written to docs/experiments/ with the dataset name and model name.
- The README/docs are updated when a command or flow changes.

10 Experiment log template (docs/experiments/EXX_name.md)

```
# EXX: <question>
Date · dataset snapshot + sha · model(s) · code commit · Kaggle notebook version
## Hypothesis
## Setup (what changed vs baseline)
## Results (table: WER / CER / switch-point / halluc / RTF, by domain & flag)
## Error analysis -(1030 worst segments, categorised)
## Decision
```

11 Risks to the schedule

Risk	Early signal	Response
The aligner struggles on noisy or overlapping recordings	P1.7 precision < 70%	Lower θ , lean on VAD fallback for those recordings, prioritise clean recordings for v1
Review is slower than expected	P2.7 < 2× speed-up	Enable spot-check batch approval for segments with confidence > 0.9
Kaggle quota exhausted	Experiments queue up	Reduce to whisper-small only; shorter runs (2k steps); drop E5
v1 test WER far above 20%	P4.5	Error analysis first: label noise (fix data), translation (E5), hallucination (more non-speech samples), accent (more data). Do not jump to bigger models until the category is known
The plain HTML/JS reviewer is slow to extend	P2.4 feedback	Reuse the transcription app's player, key map and chunk list; add the waveform as a small canvas widget

12 Immediate next steps

Changed in v1.2: see the change log.

1. You: run the pilot on Kaggle for all recordings ([07 Kaggle Runbook](#)) and send `summary.md` plus each `report.md`.
2. Together: read the zero-shot Whisper numbers and the per-recording alignment coverage; decide which recordings are usable as they are.
3. You: send the recording names so we can form speaker groups, then choose `test_v1` (2 groups, about 1.5 h) and `dev_v1` (1 group, about 50 min).
4. Me: Phase 0 on this branch, carrying the pilot code into the package layout.

Change log

Times are East Africa Time (EAT, UTC+3). Each entry names the affected section, quotes what it said before and summarises what it says now, and links to the previous text: on GitHub at the exact commit and lines, and as an archived PDF of the previous version.

Version	Date (EAT)	Commit	Summary
1.0	2026-09-24 00:36	053e09d	First LaTeX version (converted from the Markdown draft of 2026-09-23).
1.1	2026-09-24 00:45	695b407	Labelling track sized for the real corpus (11 recordings, 8 h 56 min).
1.1	2026-09-24 01:53	8a3be55	Speaker grouping step before choosing test/dev.
1.2	2026-09-24 22:26	c28ea1d	New Pilot section with results; P1.3 and P1.7 updated; new immediate next steps.
1.3	2026-10-02 00:13	this commit	Task tracking moved to GitHub Issues, with the mapping from plan items to issues.
1.4	2026-10-02 02:38	this commit	Transcription app built early; P2.2 and one risk reworded (no Gradio).

v1.4 • Pilot: transcription app row; P2.2; risks

Previous text: [v1.3 source](#), line 126 · [v1.3 PDF](#)

Before. P2.2: “Gradio shell with 3 tabs... mounted in FastAPI”. Risk: “Gradio limits the reviewer UX... Custom Gradio component for the waveform only”. No transcription app.

Now. P2.2: plain HTML/JS web shell, Transcribe section already built. Risk reworded: reuse the app’s player and chunk list, waveform as a canvas widget. New Pilot-table row for `transcribe.app`.

v1.3 • Task tracking (new)

Previous text: none (new subsection) · [v1.2 PDF](#)

Before. Owners appeared only in *Immediate next steps* (You / Me); no status tracking.

Now. Ownership and status live in GitHub Issues (#1–#18) with owner: and phase: labels; this document keeps tasks and acceptance criteria. Table maps plan items to issues.

v1.2 • Pilot: one recording before Phase 0 (new section)

Previous text: none (new section) · [v1.1 PDF](#)

Before. No pilot; the plan went straight from Foundations to the alignment pipeline.

Now. Records what was built for the single-recording pilot, the NRCCW_KSM09 alignment results, the tasks finished early (P0.3) and what is still open.

v1.2 • Phase 1, tasks P1.3 and P1.7

Previous text: [v1.1 source](#), lines 57–61 · [v1.1 PDF](#)

Before. P1.3: “an MMS implementation (windowed emissions, turn-anchored alignment, per-word scores)”; acceptance “mean word boundary error < 100 ms vs. hand labels”. P1.7: “Calibrate 0: align one real recording, hand-check 50 segments”.

Now. P1.3 wraps the pilot’s two-pass aligner; acceptance is the pilot numbers holding on every recording. P1.7 is done for NRCCW_KSM09 using transcriber timestamps instead of hand-checked segments, and repeats per recording through the report’s marker check.

v1.2 • Immediate next steps

Previous text: [v1.1 source](#), lines 215–226 · [v1.1 PDF](#)

Before. 1. Confirm decisions. 2. Start Phase 0 (P0.1–P0.5). 3. Pick about 10% of recordings for `test_v1`.

Now. 1. Kaggle pilot run on all recordings. 2. Review the numbers together. 3. Speaker groups, then test/dev. 4. Phase 0.

v1.1 • Phase 2: labelling track and speaker grouping

Previous text: [v1.0 source, line 85](#) · [v1.0 PDF](#)

Before. “review enough recordings to build test_v1 $\approx$ 3 h ($\geq$ 15 recordings, stratified), dev_v1 $\approx$ 2–3 h ($\geq$ 10 recordings), and train $\geq$ 20 h for v1.”

Now. Review the current 11 recordings (8 h 56 min): test_v1 = 2 recordings ($\approx$ 1.5 h), dev_v1 = 1 ($\approx$ 50 min), train = 8 ($\approx$ 6.5 h); long-run targets reached as audio arrives. New *speaker grouping* step (`corpus/speaker_groups.py`) before choosing test/dev.